

B ve B+ Ağaçlarının Günümüzde Kullanım Alanları

Yağmur Seda Yılmaz
02240224113

Kütüphane Sistemi Benzetmesi

B-Tree ve B+ Tree yapılarını daha somut anlamak için bir **Kütüphane Arşiv Odası** benzetmesini ele alalım.

B-Tree

Kütüphaneye girdiniz. Girişte ana panolar (kök düğüm) var: "Fen Bilimleri", "Sosyal Bilimler" vb. Bu panoların yanında **hemen o alandaki en popüler birkaç kitap da sergileniyor**. Eğer aradığınız kitap o popüler kitaplardan biriye, direkt oradan alıp çıkıyorsunuz (İç düğümde verinin kendisinin bulunması). Eğer orada değilse, sizi alt koridorlara yönlendiriyor. Alt koridorlardaki raflarda da hem yönlendirme tabelaları hem de yine bazı kitaplar var. Kitabı bulduysanız, aradığınız popüler veya merkezi bir kitaba alt katlara/raflara hiç inmeden, üst koridorlarda hemen ulaşabilirsiniz (Nokta atışı aramalarda hızlıdır).

B+ Tree

Kütüphaneye girdiğinizde, üst katlardaki ve koridorlardaki panolarda **asla kitap bulunmaz; sadece yönlendirme kartları (indeks) vardır**. "A-M arası alt katta sol koridorda", "K-L arası 3. rafta" gibi. Bütün kitaplar istisnasız en alt kattaki raflardadır (Yaprak düğümler). En büyük avantajı ise şudur: **En alt kattaki tüm kitap rafları birbirine gizli bir raylı sistemle veya koridorla bağlıdır**.

Eğer siz "Sayfa 100 ile 200 arasındaki tüm tarih kitaplarını" listelemek istiyorsanız, yukarıdaki indeksleri kullanarak 100. kitabın olduğu rafı bulursunuz. Sonra yukarıdaki panolara hiç bakmadan, **rafların arasındaki o bağlantı yolunu takip ederek** sırayla 200. kitaba kadar yürürsünüz. Aralık sorguları (örneğin "tüm ocak ayı satışları") için biçilmiş kaftandır. Üst katlara tekrar tekrar çıkıp yön sormaya gerek kalmaz. Ayrıca indeks düğümleri kitap barındırmadığı için küçüktür; tek bir panoya binlerce yönlendirme sığdırılabilir (Yüksek dallanma faktörü).

Veri Tabanlarında Kullanımı

Veri tabanları, milyonlarca veya milyarlarca satır veri arasından aradığımız bir kaydı nanosaniyeler içinde bulmak zorundadır. Verinin tamamını tek tek taramak (Full Table Scan) inanılmaz yavaş olacağından, tıpkı bir kitabın arkasındaki "fihrist/indeks" bölümü gibi yapılar kullanılır.

B-Tree ve B+ Tree, modern veri tabanlarının indeksleme altyapısını oluşturan ana yapılardır. Bu ağaçlar, veritabanı sorgularındaki **en yakın komşuyu bulma (Nearest Neighbor)** ve yüksek boyutlu verilerde benzer nesneleri arama gibi en sık kullanılan ama en maliyetli işlemleri son derece verimli hale getirir.

Dünyanın en popüler ilişkisel veri tabanları, tablo indeksleri için bu ağaç yapılarını destekler. Ancak her sistem, temel yapıyı kendi ihtiyaçlarına göre **varyasyonlar ve uzantılarla** özelleştirmiştir.

MySQL (InnoDB), tamamen **B+ Tree** mimarisini kullanır. InnoDB depolama motorunda veriler "Clustered Index" (Kümelenmiş İndeks) yapısında, doğrudan B+ Tree'nin yaprak düğümlerinde saklanır. Bu sayede ardışık veri okuma ve aralık sorguları maksimum hızda çalışır.

PostgreSQL, varsayılan indeksleme türü **B-Tree**'dir. Ancak PostgreSQL, standart B-Tree algoritmasını geliştirerek yaprak düğümlerinin birbirine bağlı olduğu, B+ Tree performansına yakın özel bir yüksek performans varyasyonu kullanır. Eşitlik ve aralık sorgularında çok güçlüdür.

Microsoft SQL Server, Oracle, IBM Db2, Informix, Sybase ASE ve SQLite ; bu dev sistemlerin tamamı, tablo indekslerini optimize etmek ve sorgu yanıt sürelerini milisaniyelere indirmek için arka planda özelleştirilmiş B+ Tree veya gelişmiş B-Tree türevlerini kullanır.

MongoDB, doküman tabanlı (JSON/BSON) bu popüler NoSQL sistemi, *WiredTiger* depolama motoru aracılığıyla indeksleme için **B-Tree** yapısını kullanır. Bu sayede rastgele erişimli tekil doküman sorgularını (nokta sorguları) çok hızlı çözer.

CouchDB ve Tokyo Cabinet, veri erişim hızını ve disk üzerindeki veri kararlılığını sağlamak amacıyla arka planda **B+ Tree** mimarisinden yararlanan diğer önemli NoSQL sistemleridir.

B-Tree, verinin kendisi veya veriye giden işaretçi (pointer) iç düğümlerde de tutulabilir. Rastgele, tekil bir veriyi (örneğin sadece **ID = 545** olan kullanıcıyı) ararken, ağacın en altına inmeden yukarıdaki düğümlerde veriyi bulup erken çıkış yapabilir.

B+ Tree'de ise, iç düğümlerde sadece yönlendirici anahtarlar bulunur; tüm veriler istisnasız en alttaki yaprak düğümlerde saklanır. Yaprak düğümler birbirine bağlı (linked list şeklinde) olduğu için, veritabanından bir aralıkta veri istenirken (örneğin "Maaşı 20.000 ile 50.000 arasındaki çalışanları getir") diskte ardışık ve kusursuz bir tarama yapar.

Dosya Sistemlerinde Kullanımı

Bir dosya sistemi için en temel zorluk, dosya içindeki rastgele bir bloğun (mantıksal adres) disk üzerindeki gerçek konumuna (fiziksel adres) hızla dönüştürülmesidir.

Tüm modern işletim sistemlerinde dosyalar dinamik olarak büyür. Bu yüzden disk blokları ardışık olmayabilir. Bu karmaşık eşlemeyi çözmek ve rastgele erişimi hızlandırmak için **B-Tree ve B+ Tree** yapıları zorunlu hale gelmiştir.

B+ Tree, özellikle meta veri (dosya adı, boyutu, izinleri) ve dizinlerin (klasör yapılarının) indekslenmesinde standart haline gelmiştir.

NTFS (Windows), ReiserFS, NSS, XFS, JFS ve ReFS sistemleri meta veri indekslemek için doğrudan B+ Tree kullanır. NTFS ayrıca güvenlik haritalandırmasında da bu yapıdan yararlanır. **BFS**, klasör dizinlerini doğrudan B+ Tree mimarisinde saklar. **EXT4 (Linux)**, dosya uzantılarını indekslemek için B+ Tree'nin özelleştirilmiş bir hali olan "**Uzantı Ağaçları**" (**Extent Trees**) yapısını tercih eder. **APFS (Apple)**, nesne kimliklerini disk konumlarına eşlemek için B+ Tree kullanır (*Tek farkı, yaprak düğümlerinde yan yana kardeş işaretçilerinin olmamasıdır*). **HAMMER (DragonFly BSD)**, değiştirilmiş, optimize bir B+ Tree yapısı kullanır.

Rastgele blok erişimini optimize etmek için doğrudan B-Tree kullanan sistemler de mevcuttur.

Apple'ın **HFS+** ve **APFS** mimarileri, Microsoft'un **NTFS** sistemi ile Linux dünyasındaki **Bcachefs, Btrfs ve AIX (jfs2)** dosya sistemleri belirli alanlarda B-Tree yapısından yararlanır.

Arama Motorları ve Büyük Veri Sistemleri

Büyük veri (Big Data) çağında veri artık megabaytlarla değil, terabaytlar ve petabaytlarla ölçülmektedir. Milyarlarca satır veya dokümandan oluşan bu devasa veri yığınlarının tamamını RAM'de (bellekte) tutmak finansal ve fiziksel olarak imkansızdır. Veri mecburen disklerde veya dağıtık sunucularda saklanır. Bu kadar büyük bir veri havuzunda geleneksel arama yöntemleri çöker. B ve B+ ağaçları, arama uzayını her adımda devasa oranlarda daraltarak sistemlerin bir "veri bataklığına" dönüşmesini engeller.

Google, Yandex veya yerel arama motorları, milyarlarca web sayfasını tarayıp aradığınız bir kelimeyi saliseler içinde önünüze getirmek için arka planda bu ağaç yapılarından yararlanır.

Arama motorları, web sitelerindeki kelimeleri ayıklayıp "Hangi kelime, hangi web sayfalarında geçiyor?" listesini tutan bir **Ters Yüz Edilmiş İndeks (Inverted Index)** mimarisi oluşturur. Bu kelime indeksleri devasa boyutlara ulaştığında, kelimelerin alfabetik veya sayısal olarak sıralı tutulması ve aranması için B+ Tree (veya gelişmiş türevleri) kullanılır. Böylece kullanıcı bir kelime arattığında, arama motoru diske binlerce kez vurmak yerine sadece 3-4 adımda o kelimenin geçtiği tüm sitelerin listesine (yaprak düğümlere) ulaşır.

Hadoop, Cassandra, HBase gibi modern büyük veri ekosistemleri, verileri diske yazarken ve okurken B-Tree/B+ Tree mimarisinden ve onların evrimleşmiş hallerini kullanır.

Büyük veride en maliyetli işlemlerden biri, belirli bir zaman aralığındaki veya belirli kriterlere uyan verilerin topluca çekilmesidir. B+ Tree'nin yaprak düğümlerinin birbirine bağlı olması, dağıtık disk blokları üzerinde sıralı tarama yapmayı oldukça kolaylaştırır.

Neden Modern Sistemlerde Tercih Edilmektedir?

Bu ağaçlar ikili arama ağacından farklı olarak derinliği değil genişliği artırmaya yöneliktir. Yani günümüzde kullanılan milyonlarca veri BST ile tutulursa veriyi arama ve bulma hem karmaşık olacak hem de uzun sürecektir. Ama B- Tree ve B+ Tree yapıları verileri daha az ve daha geniş katmanlarda tutar ve bu veriyi bulmayı kolaylaştırır. Ayrıca bu yapıda zaman karmaşıklığı tüm işlemlerde sabittir.

Bu Veri Yapıları Neden Önemlidir?

Bu yapılar sistemlerdeki en kötü senaryolarda bile tahmin edilebilir şekilde performans sergileyebilirler. Örneğin klasik bir ağaç (ör. BST) kullanılırsa ağacın dengesi bozulabilir. Yani sağa ya da sola daha ağırlık verilebilir. Böyle olunca da sistemin bağlı listeden farkı kalmaz. Bu da performansı inanılmaz derecede düşürür. Ayrıca sistemlerde kullanılan “aralık bulma” sistemini de bu yapılar sağlar. Bir alışveriş sitesini düşünelim. Fiyat aralığı istenilen şekilde ayarlanabilir. Bu yapılar olmasaydı bu işlem için ağacın yaprakları ve kökü arasında birçok kez gidip gelmek gerekirdi.

Büyük Veri Çağında Neden İhtiyaç Duyulmaktadır?

Büyük veri çağında veriler artık çok önemli bir yere sahiptir. Ama bu koskocaman verileri işlemek çok daha önemlidir. Bu verileri hızlı işleyemezsek bundan sonra pek de bir önemi kalmıyor. Bunun yanı sıra verilere hızlı ulaşmak da oldukça önemlidir. Yani bu yapılar büyük veri çağı için tam da istenen özelliklere sahiptir.

Sizce Gelecekte De Kullanılmaya Devam Edecek Midir?

Elbette bence ne kadar zaman geçerse geçsin kullanılmaya devam edebilir. Her ne kadar çok hızlı gelişen bir dünyada yaşıyor olsak da geçmişte geliştirilen ve günümüzde hala kullanılan sistemler de vardır. Sadece ilk haliyle kalmamış birçok gelişime uğramışlardır. Örneğin makine dili hala makine dilidir. Sadece insanların anlaması için çok fazla gelişime uğramıştır. bu yapılar için de aynı şey olacağını düşünüyorum. Sistem aynı kalabilir ama sürekli gelişecektir.